User Stories

בתור תושב הרשות המקומית המחובר לאזור האישי

אני רוצה לראות את כל הבקשות שהגשתי לרשויות כולל הסטטוס העדכני , תאריך היעד לטיפול, וציון עבורי האם נדרש משהו מצידי כדי לסיים את הבקשה ולסגור אותה

על מנת שאוכל לעקוב בצורה שקופה אחר התקדמות הפניות שלי ולדעת האם נדרש ממני פעולה נוספת לסגירת הטיפול

Tasks

2 -task1 Backend - API & Controller שעות

- יצירת קונטרולר שיחשוף את נקודת הקצה המאובטחת
- שליפת מספר ת.ז ישירות מתוך הclaim של הטוקן

Task2 Backend - Integration & Data Normalization // 5 שעות

- יצירת ממשק ומבנה נתונים מדומה עבור מערכת הטפסים ולקטלוג השירותים (ServiceCatalog) המדמה עבודה מול MongoDB
- ליצור קשיח אוסף של ServiceCatalog שניגש אליו ונשלוך ממנו את ההגדרות השירות לכל קוד טופס

ביצוע הנרמול:

- החלפת קוד הטופס בשם השירות, וקוד הרשות בשם הרשות

- תרגום קוד הסטטוס לשם הסטטוס שהרשות הגדירה.
- טיפול בסטטוס לא מוכר (הצגה כ"בטיפול" וכתיבה ללוג).
- actionRequired אם הסטטוס מופיע ברשימת הסטוסים שדושרשים פעולה מהתושב
- בדיקת isOverdue לחריגה מזמן הטיפול (SLA) של הרשות.
- סינון והתעלמות מקוד טופס שאינו קיים בקטלוג, לצד כתיבה ללוג.
- מיון הרשימה: בקשות הדורשות פעולה קודם, ולאחר מכן לפי תאריך העדכון האחרון מהחדש לישן.
- החזרת האובייקט המנורמל לצד הלקוח (ללא מספר ת"ז).

Task3 Frontend - Angular UI Component // שעה

- יצירת רכיב טבלה/כרטיסיות באנגולר באזור האישי.
- הוספת חיוויים בולטים לבקשות הדורשות פעולה מהתושב או שבחריגת SLA.
- טיפול במצבי קצה: תצוגת טקסט "עדיין לא הגשת בקשות" כשאין נתונים, והצגת הודעת שגיאה ידידותית עם אפשרות לנסות שוב (Retry) כשהשירות לא זמין. וידוא תמיכה ברספונסיביות, כיווניות (RTL) ונגישות (Accessibility).

Task 4: Testing שעות 2 //

- הוספת קובץ בדיקות יחידה (Unit Tests) המכיל לפחות 4 בדיקות ללוגיקת הנרמול,
- כולל מקרי קצה של סטטוס לא מוכר וחריגה מ- SLA

חלק ב'

תכנית עבודה ואינטגרציה (לשבועות הקרובים)

1. חלוקה לשלבים ואבני דרך לאורך 8 השבועות (כולל עבודת הספק)

שבועות 1-2 (אפיון טכני, חוזי עבודה והכנות):

פעילות מוביל/ה: תיאומי ממשקים מול צוות מערכת הטפסים והאזור האישי -(API Contracts).
הגדרת סכמת ה-MongoDB ל-ServiceCata
log, ואפיון סופי של מודל הנתונים והנרמול.

פעילות הספק: הקמת התשתית ב-.NET Core (ארכיטקטורת Microservices, שימוש בהזרקת תלויות וממשקים), והקמת סביבת העבודה הראשונית לצד הלקוח.

שבועות 3-4 (פיתוח ליבה ואינטגרציית מוקים - Mocks):

פעילות הספק: מימוש הלוגיקה העסקית הכוללת:

- שירות הנרמול,
- בדיקה אם עבר הזמן של הטיפול SLA,
- בדיקת actionRequired,
- מיון

- סינון
- הסרת ת"ז כדי שלא יעבור לצד הלקוח.

בגלל הפיתוח המקביל של ה-API במערכת הטפסים, הפיתוח והבדיקות יתבצעו מול Mock יציב המבוסס על חוזה העבודה המוסכם.

פעילות QA: הכנת סביבת הבדיקות והתשתית לבדיקות האוטומטיות שייצרו בהמשך.

<u>שבועות 5-6 (אינטגרציה מלאה ובדיקות מערכת):</u>

פעילות מוביל/ה + ספק:

ם סיום בניית ה-API על ידי צוות הטפסים ושחרורו לסביבת הבדיקות, יבוצע החיבור ל-API האמיתי של מערכת הטפסים, תבוצע אינטגרציה מול האזור האישי, וייערכו בדיקות מקצה לקצה.

פעילות QA: הרצת תרחישי בדיקות מקיפים (כולל זמינות, שגיאות, חריגות SLA ורספונסיביות צד לקוח).

<u>שבועות 7-8 (הקשחה,ייצוב ועלייה לייצור - Production):</u>
פעילות מוביל/ה + ספק:

- תיקון באגים

- בדיקות עומסים ואבטחת מידע,
- פריסה לסביבת Staging אחרונה,
- מעבר חי (Go-Live) לסביבת הייצור תוך ניטור צמוד.

2. ממשקים מול צוותים מקבילים והתמודדות עם API חסר.

צוות האזור האישי: הגדרת פורמט התגובה לרכיב ה-Frontend ואופן העברת מספר תעודת הזהות בתוך הטוקן בשבוע הראשון לפרויקט.

צוות מערכת הטפסים: יש להגדיר מיד בספרינט הראשון מסמך OpenAPI/Swagger משותף ל-API הבקשות לפי ת"ז. כל עוד ה-API שלהם עדיין לא מוכן, המפתחים יעבדו מול נתונים מדומים המדמים תגובות אמת כולל בדיקת מקרי קצה כמו נפילות, עיכובים או רשימה ריקה
בהתאם למבנה הנתונים שהוגדר במטלה.

שדירת המידע: הגדרת חוזה ההודעות לעדכוני סטטוס עתידיים, תוך הוצאת הנושא מ-Scope של גרסה V1 ובניית תשתית פתוחה להרחבה בגרסה הבאה.

3. תכנית QA: סוגי בדיקות, אחראים ותרחישי חובה סוגי בדיקות ואחראים:

- בדיקות יחידה (Unit Tests):
יבוצעו על ידי מפתחי הספק (כולל לוגיקת נרמול, סטטוסים

לא מוכרים וחריגות SLA).

- בדיקות אינטגרציה ו-API:
  יבוצעו על ידי בודק ה-QA והספק באמצעות Postman כדי לוודא שקריאות ה-API והתקשורת בין המערכות השונות עובדות ועוברות חלק.
- בדיקות ממשק משתמש ונגישות (UI/UX & Accessibility):
- בדיקות רספונסיביות ותמיכה ב-RTL.

תרחישי חובה לפני עלייה לייצור:

- קבלת משתמש עם בקשות מרובות מרשויות שונות, לרבות סטטוס לא מוכר (וידוא כתיבה ללוג והצגה כ"בטיפול").
- בדיקת חריגת SLA (isOverdue) ובדיקת שדות הדורשים פעילות משתמש (actionRequired).
- תרחיש נפילה/אי-זמינות של מערכת הטפסים- וידוא הצגת הודעת שגיאה ידידותית וכפתור "נסה שוב" בצד הלקוח.
- וידוא מוחלט ששדה מספר ת"ז (applicantId) אינו מועבר כלל לצד הלקוח.

4. שלושה סיכונים מרכזיים ודרכי צמצום

סיכון: איחור של צוות מערכת הטפסים באספקת ה-API האמיתי.
צמצום: עבודה קשיחה מראש מול Mock מוגדר היטב המבוסס על חוזה סגור, כך שהפיתוח והבדיקות בצד שלנו לא יעצרו.

סיכון: מערכת הטפסים איטית מאוד או נופלת, דבר שיכול לגרום לאזור האישי של התושב לטעון לאט או להיתקע.
צמצום: הוספת מנגנוני הגנה בקוד- יש להגדיר זמן הגבלה לציפייה (Timeout), הגנה שמנתקת חיבור תקוע כדי לא להפיל את המערכת (Circuit Breaker), גדרת זמן הגבלה לציפייה (Timeout), מנגנון ניתוק חיבור תקוע (Circuit Breaker), ושמירת נתונים בזיכרון זמני (Cache) כדי למנוע פניות חוזרות ונשנות למערכת המקור.

סיכון: איכות קוד ירודה או אי-עמידה בסטנדרטים מצד הספק החיצוני.
צמצום: קיום Code Reviews מידי שבוע על ידי המוביל/ה, אכיפת כללי Clean Code והגשת בדיקות יחידה כדרישת סף לאישור משימות.

חלק ג' - שאלות תכנון ואבטחה.

מידע אישי - ת.ז

במסך: השרת ימחק/יסתיר את שדה תעודת הזהות מהתשובה שהוא מחזיר לדפדפן של הלקוח.

בלוגים: איסור מוחלט על רישום מספרי ת"ז בלוגים.
ירשם בהם רק מזהה הבקשה (requestId אנונימי) או שיובצעו טשטוש למידע הרגיש.

בתקשורת: כל התקשורת בין השרתים תהיה מוצפנת ומאובטחת בהסתמכות על אימות JWT מאובטח.

נפילה או השהייה ארוכה של מערכת הטפסים שיכולה לפגוע באזור האישי

יש להגדיר זמן ציפייה קצר (Timeout) –אם המערכת לא עונה מהר, מנתקים ולא מחכים.
אם הייתה תקלה זמנית, מנסים שוב בצורה חכמה (Retry).

אם נראה שמערכת הטפסים ממש למטה יופעל מנגנון הגנה (Circuit Breaker) שיעצור את הפניות אליה כדי למנוע הצפה, ותוחזר למשתמש הודעת שגיאה מבוקרת, רשימה ריקה או מידע שמור בזיכרון (Cache), מבלי לפגוע ביציבות האזור האישי.

עדכונים בזמן אמת (שדירת המידע):

יש לחבר את המערכת להאזנה להודעות עדכון RabbitMQ ולהציג אותן למשתמש בזמן אמת במסך.

כדי שלא יהיו כפילויות, יש לבדוק תמיד את מזהה ההודעה ואת שעת העדכון האחרון (lastUpdate) –אם העדכון כבר הגיע או ישן יותר, מתעלמים ממנו ולא נדרוס מידע חדש.

4. אינדקסים ב-MongoDB ושמירת עותק בקשות:

אינדקסים לשליפה מהירה: הגדרת אינדקסים ב-ServiceCatalog ב-MongoDB על שדות מפתח כגון קוד הטופס (formCode) ומזהה הרשות (authorityId), כדי להבטיח שליפות מהירות ויעילות.

אי-שמירת עותק של בקשות התושבים: המיקרו-שירות שלנו לא ישמור עותק של בקשות התושבים ב-MongoDB. המקור הבלעדי לנתוני הבקשות הוא מערכת הטפסים. אי-שמירת עותק מקומי מונעת בעיות של סנכרון ועקביות מידע, מצמצמת סיכוני אבטחת מידע, ושומרת על פרטיות התושב.

5. ניהול הגרסאות (הוספת שדה חובה ללא שבירת צרכנים):

דרך א': פותחים גרסה חדשה ל-API (למשל v2), ומשאירים את v1 רצה במקביל לתקופת מעבר.

דרך ב': מוסיפים את השדה החדש בצורה שאינה שוברת (עם ערך ברירת מחדל) כך שגם מי שלא מכיר את השדה החדש יוכל להמשיך לעבוד כרגיל.

איך מריצים את הפרויקט ב-GitHub Codespaces

1. יצירת טוקן בדיקה

פתחו טרמינל בתיקיית הפרויקט הראשית והריצו:

node MuniServices.Api/tools/generate-dev-token.mjs

בטרמינל נוסף

עבור הרצת צד השרת

dotnet run --project MuniServices.Api/MuniServices.Api.csproj --launch-profile http

בטרמינל נוסף

עבור הרצת צד הלקוח

cd Client

npm start -- --host 0.0.0.0

אחרי שמריצים את האנגולר בנתיב הזה

https://ominous-capybara-4j57rq97xjg4fw9g-4200.app.github.dev/

יש להדביק בקונסול בדפדפן את שורות הקוד האלו על מנת שיכיר את הטוקן

sessionStorage.setItem('access_token', 'טוקן ');

location.reload();

תוקפו של הטוקן שעה

// אם יש בעיה בהרצה gh codespace ports visibility 4200:public -c ""$CODESPACE_NAME"

להפוך ports לPublic